

Bundle Unit & the Co-Simulation Flow

This note documents (1) the project's verification methodology, (2) the new `bundle_unit.sv` hardware block, and (3) how the automated co-simulation flow was extended to prove the bundler bit-for-bit against the Python golden model.

1. The verification methodology (recap)

Every hardware block in this project is checked against a **Python golden reference** (`python_ref/hdc_ref.py`). The golden model defines "correct"; the RTL must reproduce it **bit-for-bit** over many random cases. The loop is fully automated by a single `.do` script:

```
generate random cases (Python) -> .hex files (inputs + golden outputs)
-> compile RTL + testbench (vlog)
-> simulate: TB drives inputs into the RTL, compares to golden
-> PASS (all match) / FAIL (report first differing bit)
```

There is **no hand-written hardware "answer key"** — Python is the single source of truth. This note adds the bundle datapath to that loop.

Status of the two verified blocks:

Block	RTL	Golden	Harness	Result
Bind + Permute	<code>xor_permute_top.sv</code> , <code>permute_stage.sv</code>	<code>bind_permute()</code>	<code>sim/run_cosim.do</code>	PASS 1000/1000
Majority Bundle	<code>bundle_unit.sv</code>	<code>BundleAccumulator</code>	<code>sim/run_bundle_cosim.do</code>	PASS 500/500

2. What "bundling" is

In Hyperdimensional Computing, **bundling** combines several 1024-bit hypervectors into a single vector that is *similar to all of them* — it behaves like a set or an average. The standard binary implementation is a **per-bit majority vote**: for each of the 1024 bit positions, count how many input vectors have a `1` there; the output bit is `1` if that count crosses a threshold.

Analogy: stack several transparent sheets; the result shows what most sheets agree on. This is how the encoder fuses the 4 EMG channels into one "snapshot," and how training averages many examples into one class prototype.

3. The `bundle_unit.sv` design

3.1 Interface

Signal	Dir	Width	Meaning
<code>clk</code> , <code>rst_n</code>	in	1	clock, async active-low reset
<code>clear</code>	in	1	synchronous: start a fresh bundle (zero the counters)
<code>in_valid</code>	in	1	accumulate <code>in_vec_flat</code> on this clock edge
<code>in_vec_flat</code>	in	D=1024	one hypervector to add into the bundle
<code>out_vec_flat</code>	out	D=1024	combinational majority result so far
<code>n_accum_out</code>	out	32	number of vectors accumulated so far

It is a **streaming accumulator**: pulse `clear`, then assert `in_valid` once per vector. At any time, `out_vec_flat` is the bundle of everything accumulated.

3.2 How it works

- **Per-bit saturating counters.** The block keeps `D` counters, each `CNT_W` bits wide (default 6). On every accumulated vector, each counter whose input bit is `1` increments, **saturating** at `2**CNT_W - 1` (so it never wraps). A separate `n_accum` register counts how many vectors were added.
- **Majority threshold (combinational).** The output is produced by comparing each counter to half the number of accumulated vectors:

```
out[i] = (n_accum != 0) AND ( cnt[i] >= (n_accum >> 1) )
```

`n_accum >> 1` is integer floor-division (e.g. 8 for `n_accum` =16 or 17). This is the **exact** rule used by the Python `BundleAccumulator.threshold()`, so the hardware and the golden model agree by construction — including the tie behavior and the empty-bundle (`n_accum` =0 → all zeros) case.

3.3 Why this maps well to hardware

No multiplies, no floating point: just **increment, saturate, compare**. All 1024 lanes are independent and update in parallel. The counter width `CNT_W` is a deliberate design knob — narrower counters save flip-flops at the cost of earlier saturation — and is one of the parameters the project's Pareto study (Hook A) will sweep.

4. The bundle co-simulation flow

```
generate_vectors.py --bundle
uses hdc_ref.BundleAccumulator
```

python_ref/vectors/cosim_bundle/ bundle_in.hex (all input vectors) kcmt.hex (K per case) expected.hex (golden bundle)

tb_bundle_cosim.sv \$readmemh the .hex files

bundle_unit.sv (DUT) clear -> K x in_valid -> out_vec_flat

compare to expected.hex PASS / FAIL (first differing bit)

Step by step: 1. **Generate.** `generate_vectors.py --bundle` produces 500 cases. Each case bundles `K` (random, 2–16) random hypervectors with the golden `BundleAccumulator` and records the inputs, `K`, and the golden output. 2. **Variable K.** `kcmt.hex` stores `K` per case, so the test exercises many different `n_accum` values (odd and even) and therefore many threshold/tie situations. 3. **Drive.** For each case, `tb_bundle_cosim.sv` pulses `clear`, streams the `K` vectors in with `in_valid`, waits one cycle, then samples `out_vec_flat`. 4. **Compare.** The sampled output is checked against `expected.hex`. Any mismatch prints the case, `K`, `n_accum`, and the first differing bit, then fails; otherwise it prints `PASS`.

Run it (one command, from the repo root):

```
vsim -c -do sim/run_bundle_cosim.do
```

(Override the case count with the `NUM_CASES` environment variable.)

5. Result

```
tb_bundle_cosim: majority bundle vs Python golden
CASES = 500 (D=1024, CNT_W=6)
...
PASS: all 500 bundle cases match the Python golden bit-for-bit.
Errors: 0, Warnings: 0
```

Verified on ModelSim SE-64 10.6e. The bundler is now a trusted, golden-matched building block.

6. Where this fits, and what's next

The HDC core now has **two of its primitives verified**: bind+permute and bundle. Remaining blocks, each to be built and folded into this same harness:

- `popcount_am.sv` — masked Hamming distance + nearest-prototype decision (the "which gesture?" step). Golden already exists (`hamming` , `classify`).
- `encoder_top.sv` — chains item-memory lookups, bind, permute, and bundle into the full record/n-gram encoder.
- `hdc_stream_wrapper.sv` — AXI-Stream + DMA front-end for streaming on Zynq.

Because the Python golden for each already lives in `hdc_ref.py` , extending the co-sim to them follows the exact recipe shown here: add a `--<block>` exporter, a `tb_<block>_cosim.sv` , and a `sim/run_<block>_cosim.do` .

Summary: `bundle_unit.sv` implements per-bit saturating-counter majority voting that matches the Python `BundleAccumulator` exactly, and it now passes the automated bit-exact co-simulation (500/500 cases) under the same Python-is-truth methodology used for the bind+permute datapath.